



福建師範大學
FUJIAN NORMAL UNIVERSITY

《机器学习》

【第四章-决策树与随机森林】

集成学习

黄锐泓



本节要点

1. 偏差与方差
2. Bagging与随机森林
3. Boosting与提升树
4. Stacking





本节概览

什么是集成学习？

不是算法，是一种思想。

融合（集成）多个模型，产生一个更为强大的模型

集成学习的作用？

降低原模型（单个模型）的方差和偏差

典型的集成学习的框架包括Bagging、Boosting、Stacking





本节概览

什么是集成学习？

不是算法，是一种思想。

融合（集成）多个模型，产生一个更优模型

集成学习的作用？

降低原模型（单个模型）的方差和偏差

典型的集成学习的框架包括Bagging、Boosting、Stacking

三个臭皮匠顶一个诸葛亮





偏差 (Bias) 与方差 (Variance)

定义

偏差 (Bias) : 预测值/期望值和真实值之间的误差。

方差 (Variance) : 预测值之间的离散程度, 也就是离其期望值的距离。方差越大, 数据的分布越分散。

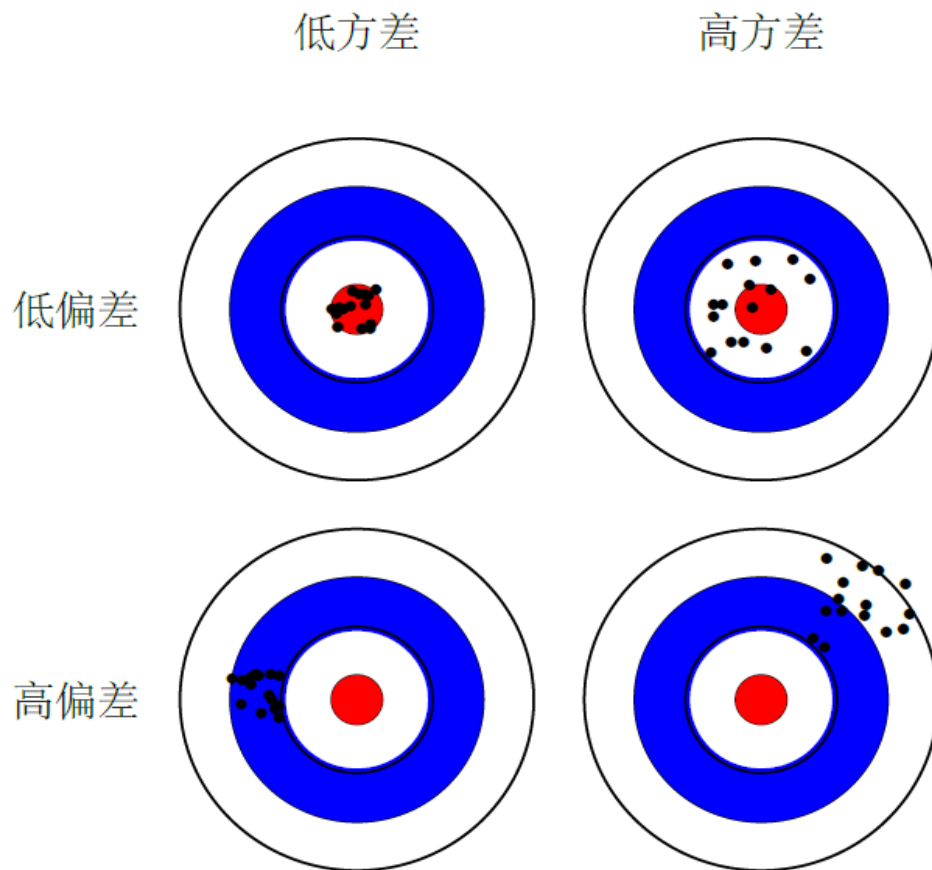
偏差和方差分别从两个方面来描述我们学习到的模型与真实模型之间的差距。





偏差 (Bias) 与方差 (Variance)

如图，红色的圆心代表理想的优化目标，黑色的点代表在不同的采样集合上训练模型的优化结果。可以看到左边一列低方差的优化结果要比右边一列高方差的优化结果更为集中，上边一行低偏差的优化结果要比下边一行高偏差的优化结果更靠近中心





偏差 (Bias) 与方差 (Variance)

数学背景

符号定义:

x 为测试样本,

D 为训练数据集,

y_D 为测试数据集 x 的标签 (采样得到),

y 为测试数据集 x 的真实标签,

$f(x; D)$ 为训练数据集 D 上学得模型 f 对于 x 的输出,

$\bar{f}(x)$ 模型 f 对 x 的期望预测输出





偏差 (Bias) 与方差 (Variance)

数学背景

1. 在一个训练集 D 上模型 f 对测试样本 x 的预测输出为 $f(x; D)$, 那么学习算法 f 对测试样本 x 的期望预测为:

$$\bar{f}(x) = E_D[f(x; D)]$$



偏差 (Bias) 与方差 (Variance)



数学背景

1. 在一个训练集 D 上模型 f 对测试样本 x 的预测输出为 $f(x; D)$, 那么学习算法 f 对测试样本 x 的期望预测为:

$$\bar{f}(x) = E_D[f(x; D)]$$

此处的期望预测是针对不同数据集 D , f 对 x 的预测值取其期望 (平均预测)。





偏差 (Bias) 与方差 (Variance)

数学背景

1. 在一个训练集 D 上模型 f 对测试样本 x 的预测输出为 $f(x; D)$, 那么学习算法

f 对测试样本 x 的期望预测为: $\bar{f}(x) = E_D[f(x; D)]$

2. 使用样本数相同的不同的训练集产生的方差:

$$var(x) = E_D[(f(x; D) - \bar{f}(x))^2]$$





偏差 (Bias) 与方差 (Variance)

数学背景

1. 在一个训练集 D 上模型 f 对测试样本 x 的预测输出为 $f(x; D)$, 那么学习算法 f 对测试样本 x 的期望预测为: $\bar{f}(x) = E_D[f(x; D)]$

2. 使用样本数相同的不同的训练集产生的方差:

$$var(x) = E_D[(f(x; D) - \bar{f}(x))^2]$$

3. 采样误差, 也称为噪声, 可以表示为:

$$\varepsilon^2 = E_D[(y_D - y)^2]$$

采样误差是指数据采集是产生的误差, 如标记错误。
通常情况下我们假设噪声服从高斯分布 $\mathcal{N}(0, \sigma^2)$ 。因此, 噪声的期望为零, 即 $E_D[y_D - y] = 0$ 。





偏差 (Bias) 与方差 (Variance)

数学背景

1. 在一个训练集 D 上模型 f 对测试样本 x 的预测输出为 $f(x; D)$, 那么学习算法 f 对测试样本 x 的期望预测为: $\bar{f}(x) = E_D[f(x; D)]$

2. 使用样本数相同的不同的训练集产生的方差:

$$var(x) = E_D[(f(x; D) - \bar{f}(x))^2]$$

3. 采样误差, 也称为噪声, 可以表示为: $\varepsilon^2 = E_D[(y_D - y)^2]$

4. 期望输出与真实标签的差别称为**偏差**, 即:

$$bias(x) = E_D[(\bar{f}(x) - y)^2]$$





偏差 (Bias) 与方差 (Variance)

数学背景

我们实际优化的目标是为了让模型的预测输出 $f(x; D)$ 与测试数据集标签 y_D 差异最小，即最小化

$$E_D[(f(x; D) - y_D)^2]$$





偏差 (Bias) 与方差 (Variance)

推导一下 $E_D[(f(x; D) - y_D)^2]$

$$= E_D[(f(x; D) - \bar{f}(x) + \bar{f}(x) - y_D)^2]$$

$$= E_D[(f(x; D) - \bar{f}(x))^2] + E_D[(\bar{f}(x) - y_D)^2]$$

$$+ E_D[2(f(x; D) - \bar{f}(x))(\bar{f}(x) - y_D)] \longrightarrow \text{此项为0}$$

$$= E_D[(f(x; D) - \bar{f}(x))^2] + E_D[(\bar{f}(x) - y_D)^2]$$

$$= E_D[(f(x; D) - \bar{f}(x))^2] + E_D[(\bar{f}(x) - y + y - y_D)^2]$$

噪声期望为0，
因此，此项为0

$$= E_D[(f(x; D) - \bar{f}(x))^2] + E_D[(\bar{f}(x) - y)^2] + E_D[(y - y_D)^2] + 2E_D[(\bar{f}(x) - y)(y - y_D)]$$

$$= E_D[(f(x; D) - \bar{f}(x))^2] + E_D[(\bar{f}(x) - y)^2] + E_D[(y - y_D)^2]$$





偏差 (Bias) 与方差 (Variance)

经上述推导可得：

$$\begin{aligned} E_D[(f(x; D) - y_D)^2] &= E_D[(f(x; D) - \bar{f}(x))^2] + E_D[(\bar{f}(x) - y)^2] + E_D[(y - y_D)^2] \\ &= \text{var}(x) + \text{bias}(x) + \varepsilon^2 \end{aligned}$$

可见，泛化误差是由偏差、方差与数据噪声之和





偏差 (Bias) 与方差 (Variance)

$$bias(x) + var(x) + \varepsilon^2$$

由于 ε^2 是一个常量，优化的最终目的是降低模型的方差及偏差。

方差越小，说明不同的采样分布D下，模型的泛化能力大致相当，侧面反应了模型没有发生过拟合。

偏差越小，说明模型对样本预测的越准，模型的拟合能力越好。

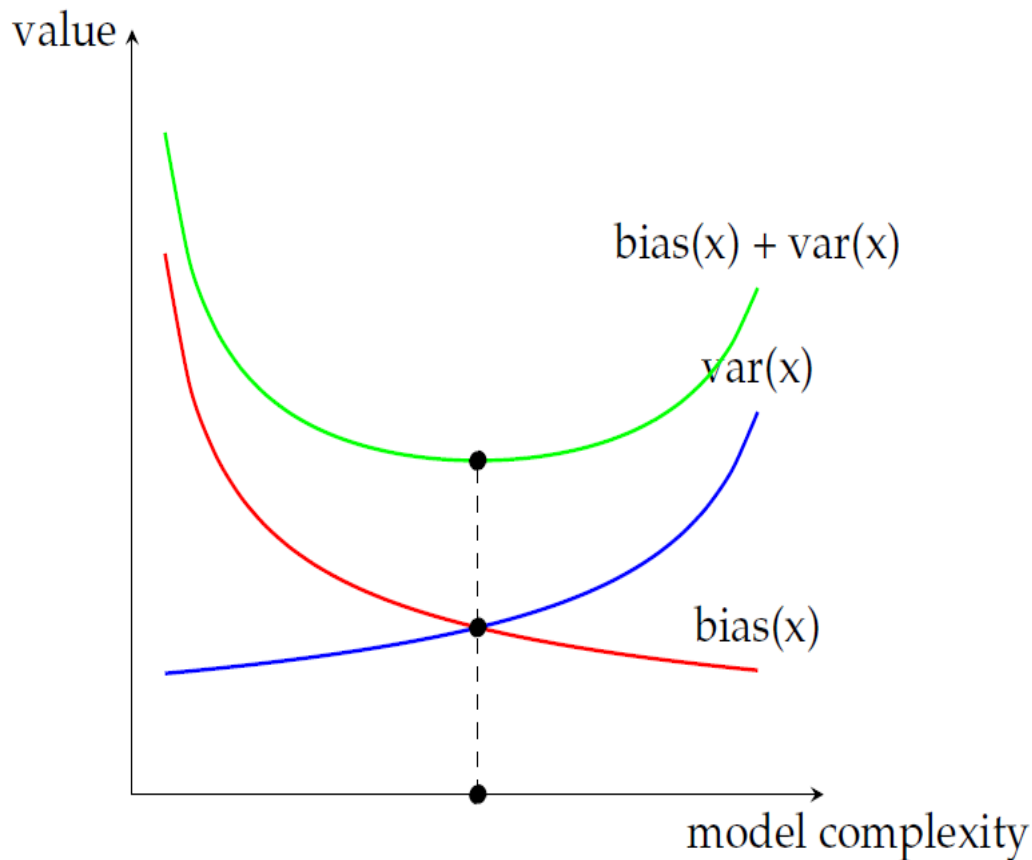




偏差 (Bias) 与方差 (Variance)

偏差-方差窘境

实际在选择模型时，随着模型复杂度的增加，模型的偏差越来越小，而方差会越来越大。如图所示，存在某时刻，模型的方差和偏差之和最小，此时模型性能在误差及泛化能力方面达到最优。



Bagging (Bootstrap aggregation)



1. 算法思路

- a) 从原始的样本集合采样（有放回采样），得到若干个大小相同的样本集合
- b) 在每个样本集合上分别训练一个模型
- c) 用投票法进行预测

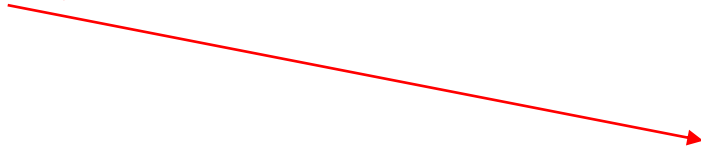




Bagging (Bootstrap aggregation)

1. 算法思路

- a) 从原始的样本集合采样（**有放回采样**），得到若干个大小相同的样本集合
- b) 在每个样本集合上分别训练一个模型
- c) 用**投票法**进行预测



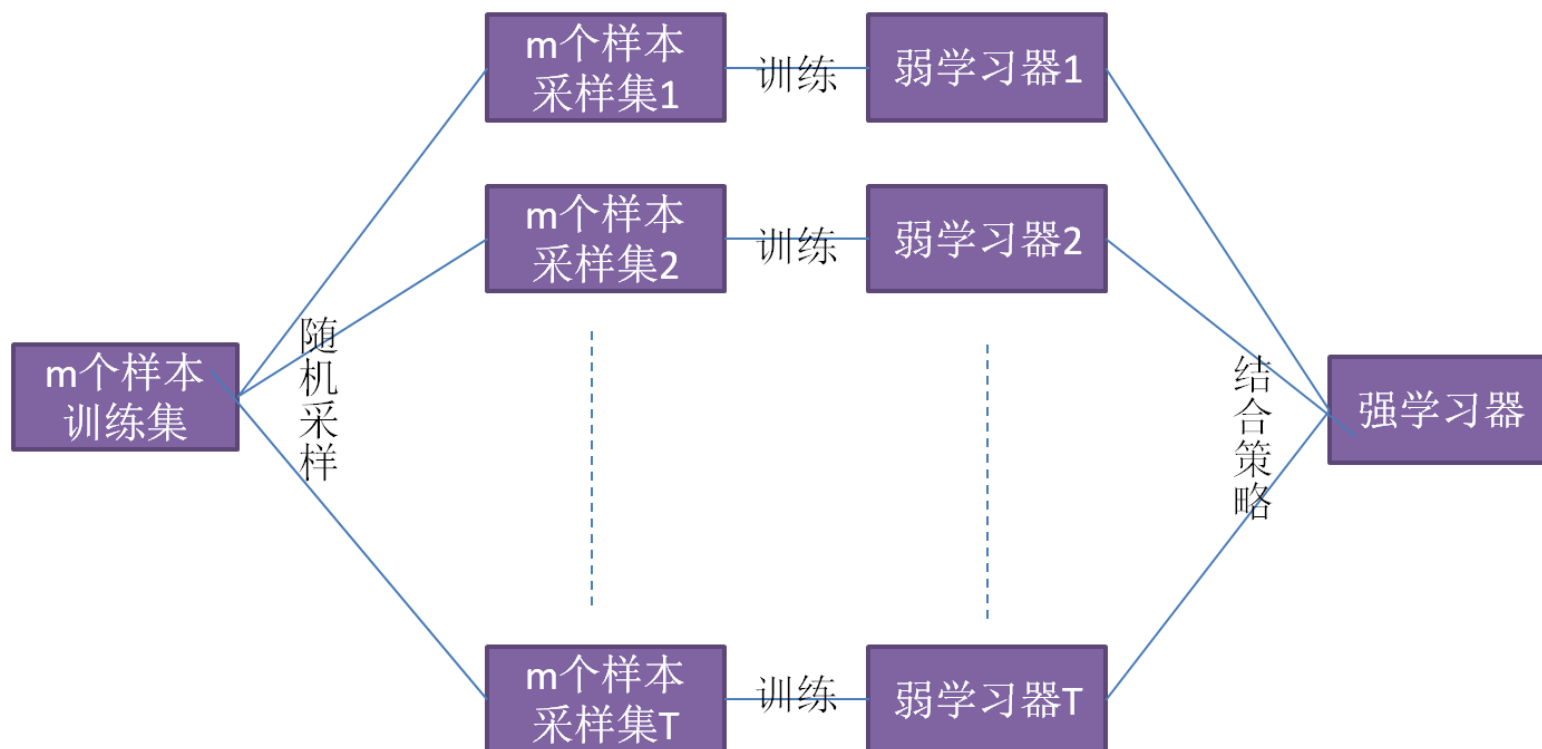
“少数服从多数”





Bagging (Bootstrap aggregation)

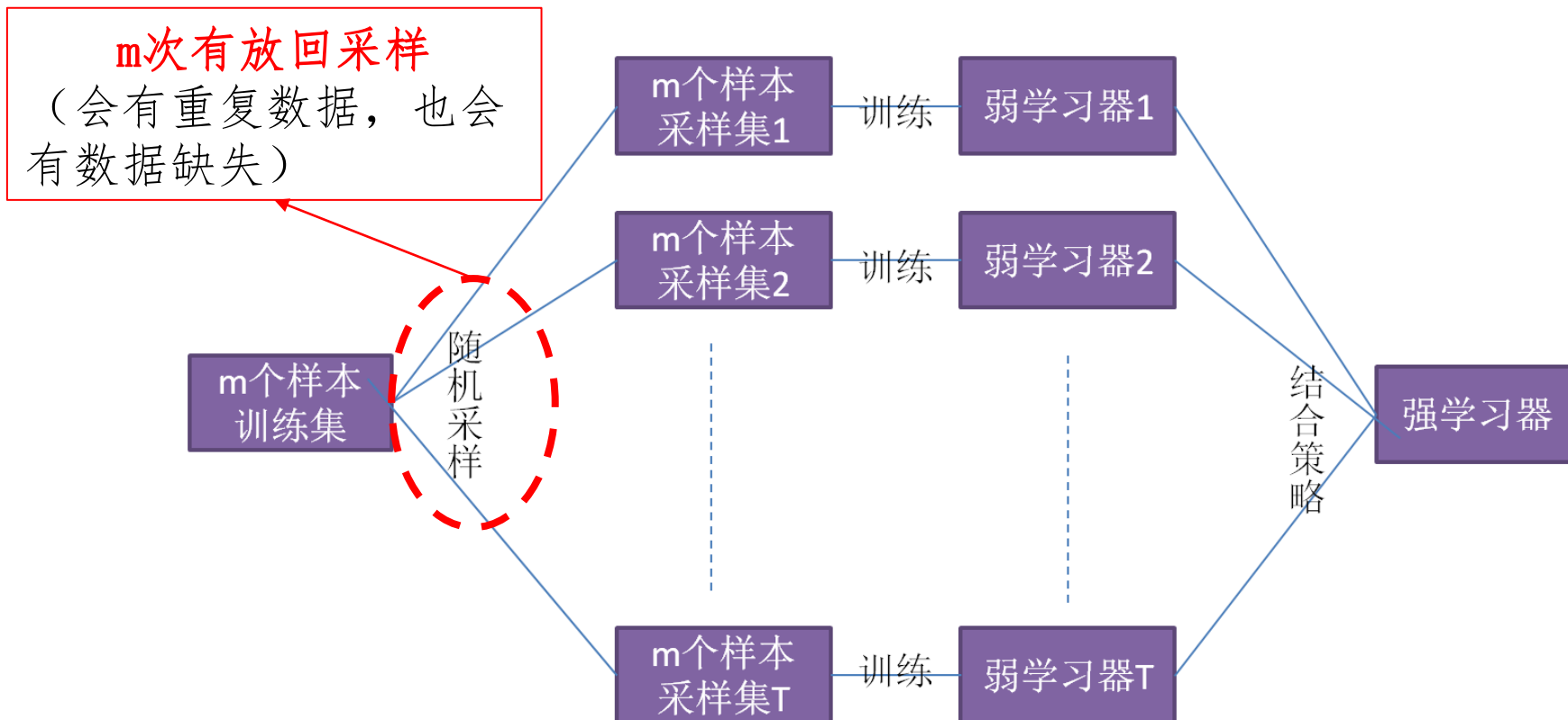
1. 算法思路





Bagging (Bootstrap aggregation)

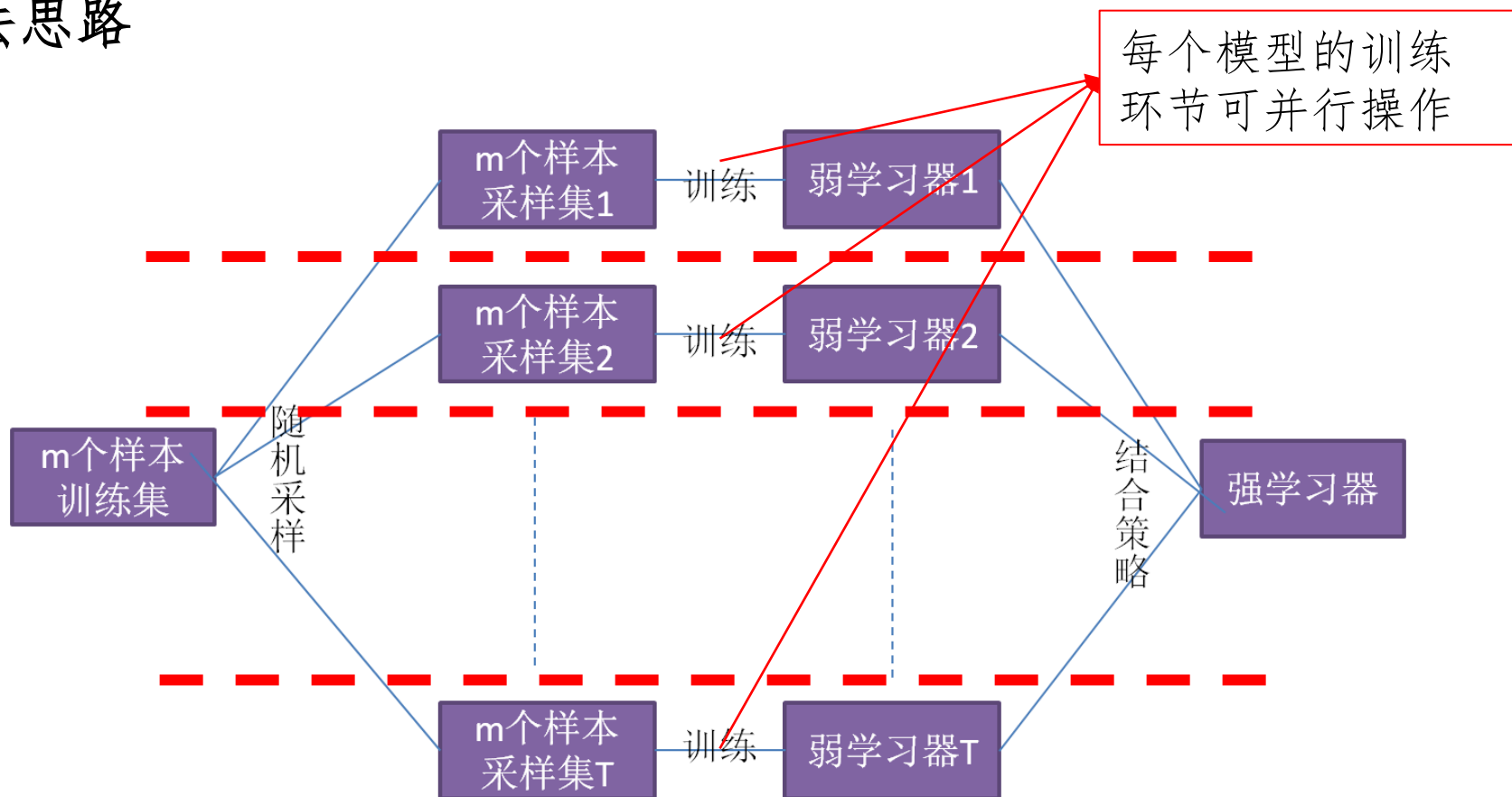
1. 算法思路





Bagging (Bootstrap aggregation)

1. 算法思路





Bagging (Bootstrap aggregation)

1. 算法思路

2. 数学背景

m 个样本，**某一样本**在一次采样中没被抽到的概率为 $1 - \frac{1}{m}$ ，连续 m 次采样都

没被抽到的概率为 $(1 - \frac{1}{m})^m$ 。

$$\lim_{m \rightarrow \infty} (1 - \frac{1}{m})^m = \frac{1}{e} \approx 0.368$$

所以说该采样方法造成的数据缺失大约为1/3





Bagging (Bootstrap aggregation)

1. 算法思路

2. 数学背景

估算集成模型 $F(\vec{x})$ 的偏差和方差

假设每个模型都是独立同分布的随机变量 $f_{D_i}(\vec{x})$ ，方差为 σ^2 ，期望/均值为 μ ，

两两模型间相关系数为 ρ ，集成模型 $F(\vec{x})$ 的期望为：

$$E[F(\vec{x})] = \frac{1}{T} E \left[\sum_{t=1}^T f_{D_t}(\vec{x}) \right] = \frac{1}{T} \sum_{t=1}^T E[f_{D_t}(\vec{x})] = \mu$$

相同





Bagging (Bootstrap aggregation)

1. 算法思路

2. 数学背景

估算集成模型 $F(\vec{x})$ 的偏差和方差

$$E[F(\vec{x})] = \frac{1}{T} E \left[\sum_{t=1}^T f_{D_t}(\vec{x}) \right] = \frac{1}{T} \sum_{t=1}^T E[f_{D_t}(\vec{x})] = \mu$$

可见，集成模型 $F(\vec{x})$ 的期望与每个独立模型 $f_{D_i}(\vec{x})$ 相同，所以**偏差相同**。





Bagging (Bootstrap aggregation)

1. 算法思路

2. 数学背景

估算集成模型 $F(\vec{x})$ 的偏差和方差

$$E[F(\vec{x})] = \frac{1}{T} E \left[\sum_{t=1}^T f_{D_t}(\vec{x}) \right] = \frac{1}{T} \sum_{t=1}^T E[f_{D_t}(\vec{x})] = \mu$$

回忆一下偏差的定义。**偏差是期望值和真实值之间的误差。期望值相同，真实值不变，自然偏差相同**

可见，集成模型 $F(\vec{x})$ 的期望与每个独立模型 $f_{D_i}(\vec{x})$ 相同，所以**偏差相同**。





Bagging (Bootstrap aggregation)

1. 算法思路

2. 数学背景

那么集成模型 $F(\vec{x})$ 的方差情况如何？

$$\begin{aligned}\text{var}(F(\vec{x})) &= \text{var}\left(\sum_{t=1}^T \frac{1}{T} f_{D_t}(\vec{x})\right) = \frac{2}{T^2} \sum_{i=1}^T \sum_{j=i}^T \text{cov}(f_{D_i}(\vec{x}), f_{D_j}(\vec{x})) \\ &= \frac{1}{T^2} \sum_{i=1}^T \text{var}(f_{D_i}(\vec{x})) + \frac{2}{T^2} \sum_{i=1}^{T-1} \sum_{j=i+1}^T \text{cov}(f_{D_i}(\vec{x}), f_{D_j}(\vec{x})) \\ &= \frac{1}{T^2} \sum_{i=1}^T \sigma^2 + \frac{2}{T^2} \sum_{i=1}^{T-1} \sum_{j=i+1}^T \rho \sigma^2\end{aligned}$$





Bagging (Bootstrap aggregation)

1. 算法思路

2. 数学背景

那么集成模型 $F(\vec{x})$ 的方差情况如何？

$$\text{var}(F(\vec{x})) = \frac{1}{T^2} \sum_{i=1}^T \sigma^2 + \frac{2}{T^2} \sum_{i=1}^{T-1} \sum_{j=i+1}^T \rho \sigma^2$$

化简可得：

$$\text{var}(F(\vec{x})) = \left(\rho + \frac{1-\rho}{T} \right) \sigma^2$$





Bagging (Bootstrap aggregation)

1. 算法思路

2. 数学背景

那么集成模型 $F(\vec{x})$ 的方差情况如何？

$$\text{var}(F(\vec{x})) = \left(\rho + \frac{1-\rho}{T} \right) \sigma^2$$

两个极端情况：

1. 所有的模型 $f_{D_j}(\vec{x})$ 相互独立。此时 $\rho=0$ ，即模型间相关系数为0， $\text{var}(F(\vec{x})) = \frac{\sigma^2}{T}$
2. 所有的模型 $f_{D_j}(\vec{x})$ 都相等。此时 $\rho=1$ ，即模型间相关系数为1， $\text{var}(F(\vec{x})) = \sigma^2$

通常情况介于二者之间，因此 $\text{var}(F(\vec{x}))$ 介于 $\frac{\sigma^2}{T}$ 与 σ^2 之间





Bagging (Bootstrap aggregation)

1. 算法思路

2. 数学背景

由上述的推导可知

集成模型 $F(\bar{x})$ 的方差在 $\frac{\sigma^2}{T}$ 与 σ^2 之间，偏差为 μ

因此，**Bagging**缩小了模型的方差。





随机森林(Random Forest)

1. 定义

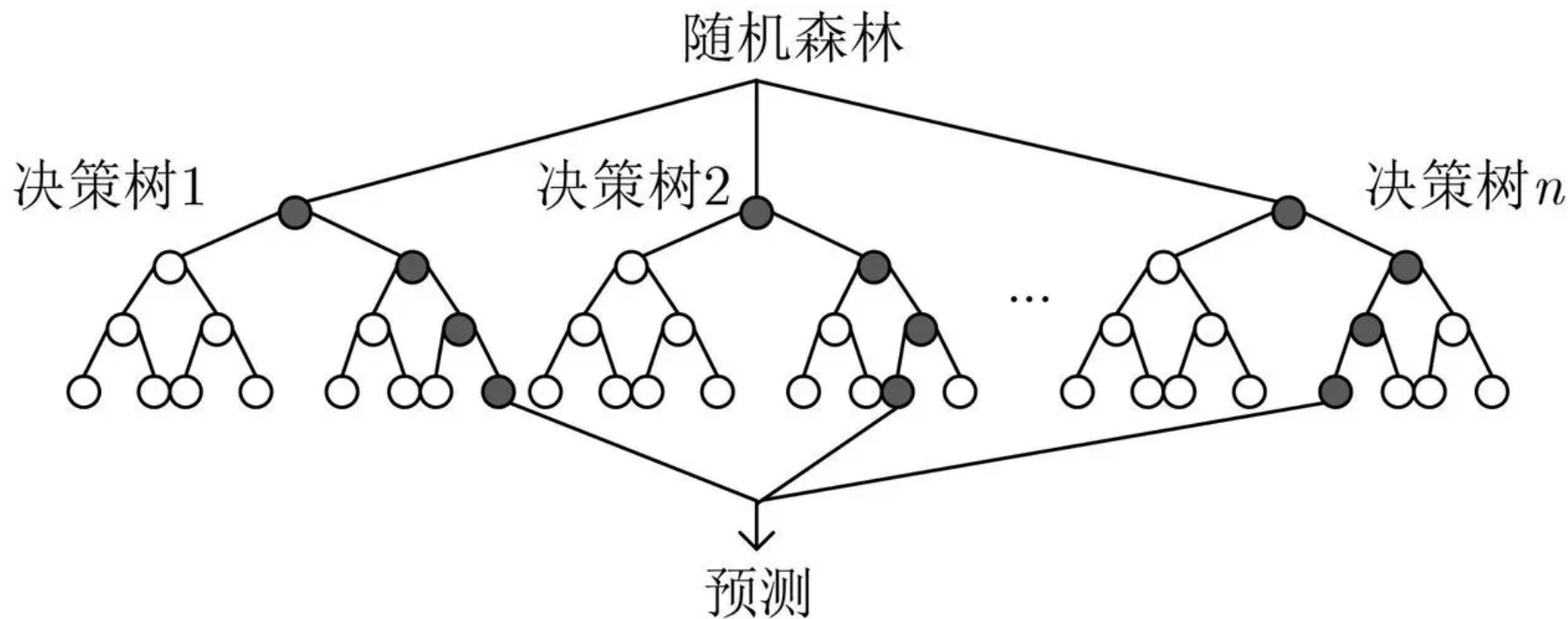
随机森林是Bagging的一个扩展变体。随机森林在以决策树为基学习器构建Bagging的基础上，进一步在决策树的训练过程中引入了随机属性选择。

简单点说，当你有很多**决策树**的时候，你就有了一片**随机森林**





随机森林 (Random Forest)





随机森林(Random Forest)

1. 定义

2. 算法思路

随机森林的做法是在Bagging采样得到的样本集合的基础上，随机从中挑选出k个属性再组成新的数据集后再训练决策树。最后训练T棵树进行集成。





随机森林(Random Forest)

1. 定义

2. 算法思路

3. 优势

a) 集成之后的模型不易过拟合，泛化能力大为增强

b) 易于实现、易于并行



Boosting



1. 算法思路

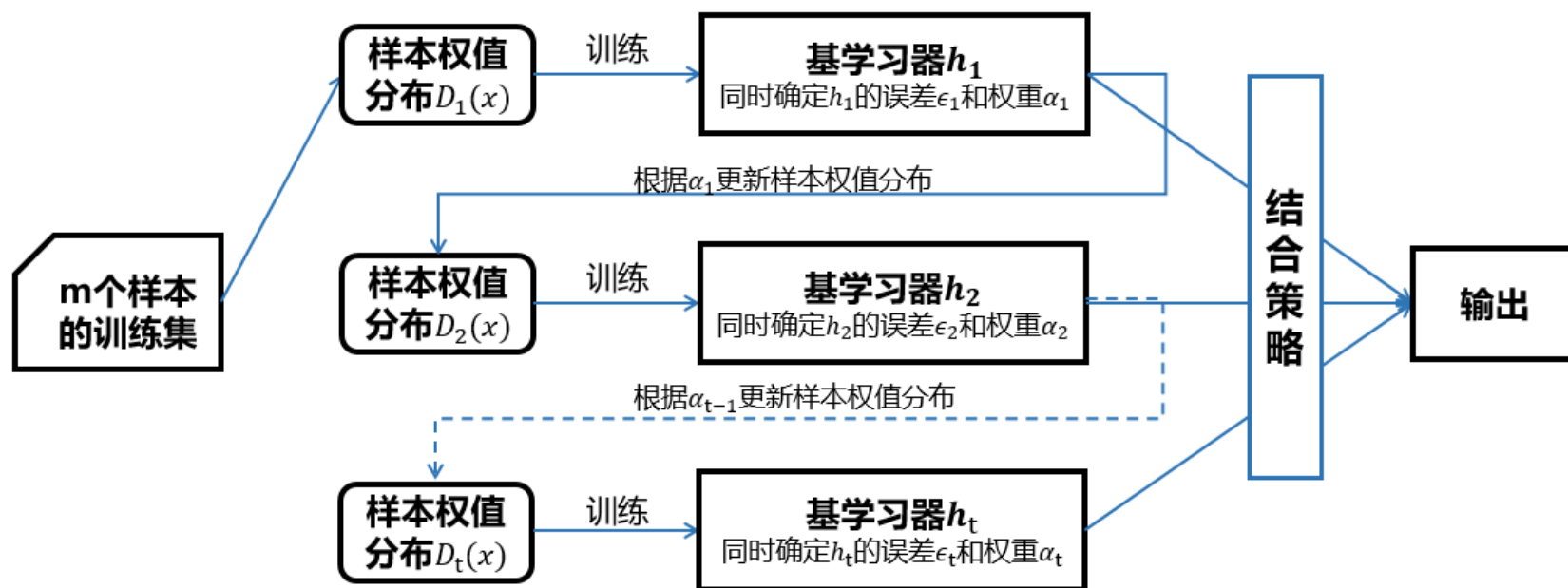
- a) 使用初始权重从训练集中训练出一个弱学习器。
- b) 根据弱学习器的学习误差率来更新样本的权重，**着重考虑被弱学习器错误分类的样本。**
- c) 如此循环，直到得到指定数量的学习器，再通过结合策略进行整合，得到最终的强学习器。



Boosting



1. 算法思路



Boosting



1. 算法思路

2. AdaBoost (Adaptive Boosting)

AdaBost是一种前向优化算法：从前向后，每次只优化一个子模型并估计其系数。每一步的优化都依赖于上一步的结果。

数学背景——略

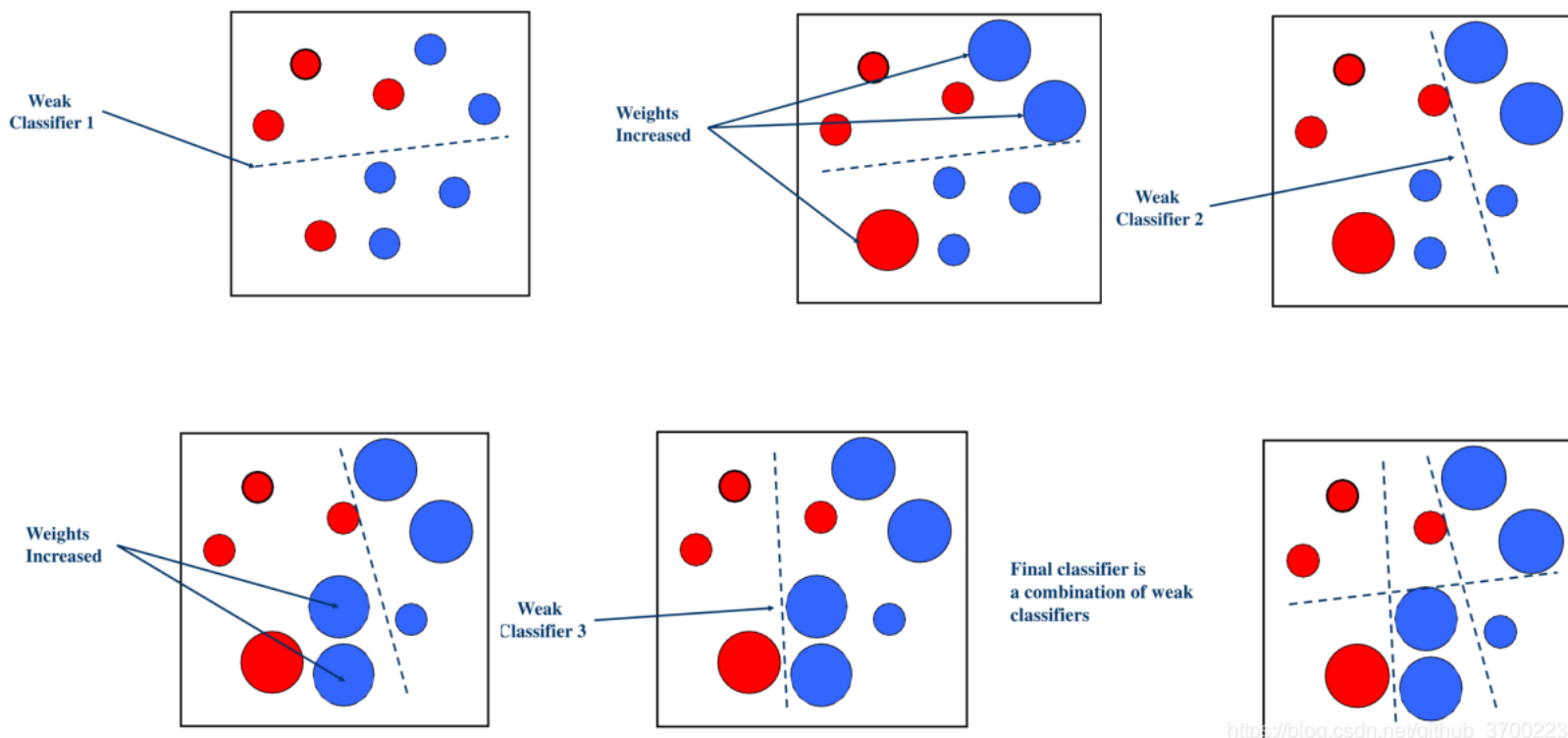


Boosting



1. 算法思路

2. AdaBoost



http://blog.csdn.net/qinub_37002238



Boosting



1. 算法思路

2. AdaBoost

总结

AdaBoost算法每次迭代关注上一步被分类错误的样本，说明AdaBoost算法着重优化的是**偏差**，对方差的优化不明显但仍有参考意义，集成模型的方差与单模型基本相同。



提升树



1. 定义

基模型为决策树的Boosting算法称为提升树。



提升树



1. 定义

基模型为决策树的Boosting算法称为提升树。

联想一下
随机森林

随机森林则是在Bagging采样得到的样本集合的基础上



提升树



1. 定义

基模型为决策树的Boosting算法称为提升树。

通常提升树以CART算法作为基模型决策树的训练方法。典型的提升树算法有GBDT、XGBOOST等。提升树有着**可解释性强**、**伸缩不变性**（无需对特征进行归一化）、**对异常样本不敏感**等优点，被认为是最好的机器学习算法之一，在工业界有着广泛的应用。



提升树



1. 定义

2. 典型案例

a. 残差提升树

残差 r : 样本的真实目标值 y 与模型预测值之差 $r = y - f(\vec{x})$

平方差损失函数

$$\begin{aligned} L(y, F_{t+1}(\vec{x})) &= L(y, F_t(\vec{x}) + f_{t+1}(\vec{x})) \\ &= (y - F_t(\vec{x}) - f_{t+1}(\vec{x}))^2 \\ &= (r - f_{t+1}(\vec{x}))^2 \end{aligned}$$



提升树



1. 定义

2. 典型案例

a. 残差提升树

残差 r : 样本的真实目标值 y 与模型预测值之差 $r = y - f(\vec{x})$

平方差损失函数

$$\begin{aligned} L(y, F_{t+1}(\vec{x})) &= L(y, F_t(\vec{x}) + f_{t+1}(\vec{x})) \\ &= (y - F_t(\vec{x}) - f_{t+1}(\vec{x}))^2 \\ &= (r - f_{t+1}(\vec{x}))^2 \end{aligned}$$

$\rightarrow F_t(\vec{x})$ 的残差



提升树



1. 定义

2. 典型案例

a. 残差提升树

b. 梯度提升树 (GBDT)

梯度提升树的整体结构与残差提升树类似。不同的是，残差提升树拟合的是样本的真实值与当前已训练好的模型的预测值之间的残差，而梯度提升树拟合的则是损失函数对当前已训好模型的负梯度。

$$-\left[\frac{\partial L(y, F(\vec{x}))}{\partial F(\vec{x})}\right]_{F(\vec{x})=F_{t-1}(\vec{x})} \quad \text{其中, } F_{t-1}(\vec{x}) = \sum_{i=0}^{t-1} f_i(\vec{x})$$



提升树



梯度提升树算法大致流程：

输入：样本集合 $D = \{(\vec{x}_1, y_1), (\vec{x}_2, y_2), \dots, (\vec{x}_m, y_m)\}$ ；决策树生成算法

输出：梯度提升树

- 初始化 $F_0(\vec{x})$

For $t = \{1, 2, \dots, T\}$

- 对每个样本计算损失函数 L 关于当前模型 $F(\vec{x}_i)$ 的负梯度
- 以负梯度为拟合对象，构建一棵决策树，假设其参数为 $\vec{\omega}_t$
- 构建好的决策树将样本集合划分为 J 个子集，每个叶节点是一个子集，记为 R_{tj} , $j = 1, 2, \dots, J$
- 估计每个叶节点的预测值
- 模型更新

End for

Return $F_t(\vec{x})$

$$-\left[\frac{\partial L(y, F(\vec{x}))}{\partial F(\vec{x})}\right]_{F(\vec{x})=F_{t-1}(\vec{x})}$$

$$\vec{\omega}_t = \underset{\vec{\omega}}{\operatorname{argmin}} \sum_{i=1}^m [\hat{y}_i - f_i(\vec{x}_i)]^2$$



提升树



1. 定义

2. 典型案例

a. 残差提升树

b. 梯度提升树 (GBDT)

c. XGBoost

XGBoost通过**正则化项**来抑制模型的复杂度，以缓解过拟合。

$$\Omega(f_t) = \gamma J + \lambda \frac{1}{2} \|\vec{\omega}\|_2^2 = \gamma J + \frac{1}{2} \lambda \sum_{j=1}^J \omega_j^2$$

J 为叶子节点数目， $\vec{\omega}$ 为每个叶节点的预测值



提升树



1. 定义

2. 典型案例

a. 残差提升树

b. 梯度提升树 (GBDT)

c. XGBoost $\Omega(f_t) = \gamma J + \lambda \frac{1}{2} \|\vec{\omega}\|_2^2 = \gamma J + \frac{1}{2} \lambda \sum_{j=1}^J \omega_j^2$

因此，XGBoost的损失函数可以表示为：

$$L_t = \sum_{i=1}^m l(y_i, F_t(\vec{x}_i)) + \Omega(f) = \sum_{i=1}^m l(y_i, F_{t-1}(\vec{x}_i) + f_t(\vec{x}_i)) + \Omega(f)$$

新增的正则项

与残差提升树类似

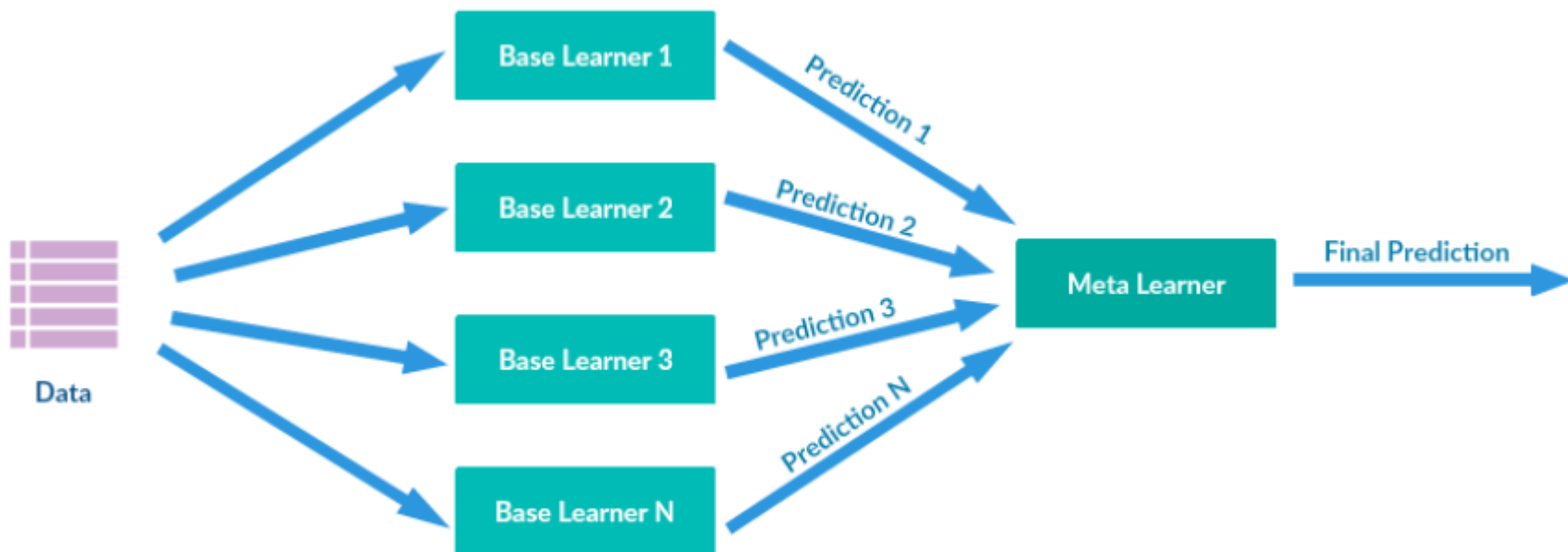




Stacking——模型堆叠

定义

用不同的子模型对输入提取不同的特征，然后拼接成一个特征向量，得到原始样本在特征空间的表示，然后在特征空间再训练一个学习器进行预测。





小结——Bagging vs Boosting

	Bagging	Boosting
采样策略	Bootstrap	训练集不变，改变每个模型样本分布的权重
权重	均匀采样，样本权重相等	提高上一轮数据中被错误分类的样本的权重，让分类器关注错误分类的样本
集成方式	子模型权重相等	误差小的模型权重大
计算方式	并行计算，可加快计算速度	模型串联，前一个模型的输出作为后一个模型的输入
特点	关注模型的稳定性，减小方差	减小偏差



小结



1. 偏差与方差的定义
2. Bagging与随机森林
3. Boosting与提升树
4. Stacking





谢谢！

